



Go from Data to Strategy: Tepper School of Business

How to Grid Search Hyperparameters for PyTorch Models

by **Adrian Tam** on April 8, 2023 in **Deep Learning with PyTorch**  6

 Share  Share  Post

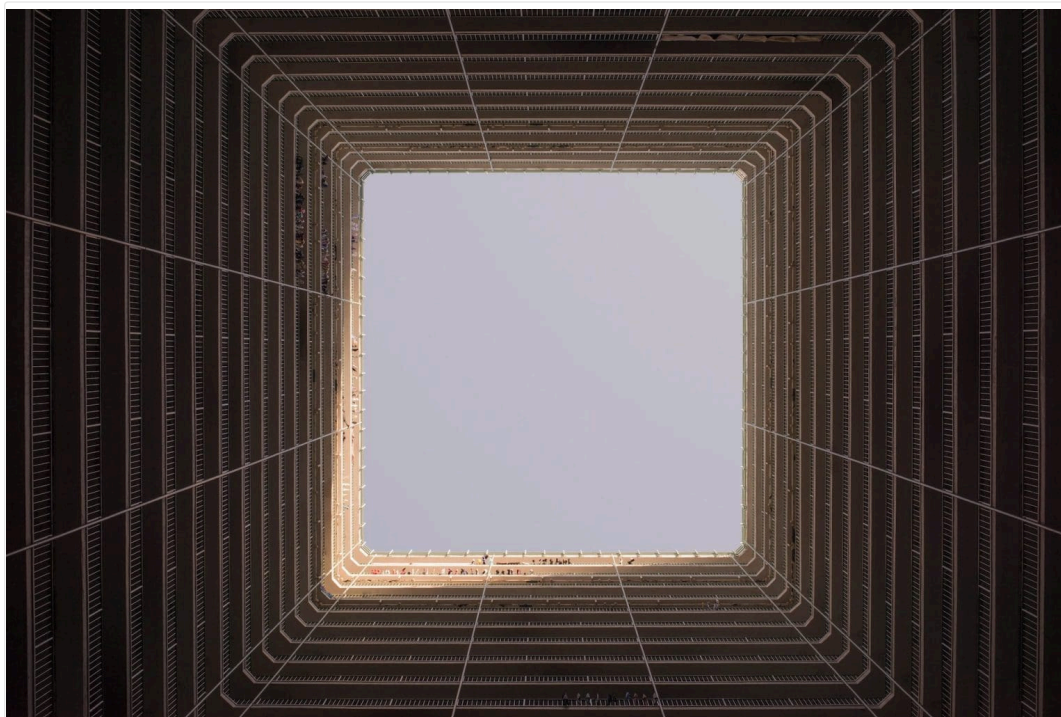
The “weights” of a neural network is referred as “parameters” in PyTorch code and it is fine-tuned by optimizer during training. On the contrary, hyperparameters are the parameters of a neural network that is fixed by design and not tuned by training. Examples are the number of hidden layers and the choice of activation functions. Hyperparameter optimization is a big part of deep learning. The reason is that neural networks are notoriously difficult to configure, and a lot of parameters need to be set. On top of that, individual models can be very slow to train.

In this post, you will discover how to use the grid search capability from the scikit-learn Python machine learning library to tune the hyperparameters of PyTorch deep learning models. After reading this post, you will know:

- How to wrap PyTorch models for use in scikit-learn and how to use grid search
- How to grid search common neural network parameters, such as learning rate, dropout rate, epochs, and number of neurons
- How to define your own hyperparameter tuning experiments on your own projects

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Let's get started.



How to Grid Search Hyperparameters for PyTorch Models
Photo by [brandon siu](#). Some rights reserved.

Overview

In this post, you will see how to use the scikit-learn grid search capability with a suite of examples that you can copy-and-paste into your own project as a starting point. Below is a list of the topics we are going to cover:

- How to use PyTorch models in scikit-learn
- How to use grid search in scikit-learn
- How to tune batch size and training epochs
- How to tune optimization algorithms
- How to tune learning rate and momentum
- How to tune network weight initialization
- How to tune activation functions
- How to tune dropout regularization
- How to tune the number of neurons in the hidden layer

How to Use PyTorch Models in scikit-learn

PyTorch models can be used in scikit-learn if wrapped with skorch. This is to leverage the duck-typing nature of Python to make the PyTorch model provide similar API as a scikit-learn model, so everything in scikit-learn can work along. In skorch, there are `NeuralNetClassifier` for classification neural networks and `NeuralNetRegressor` for regression neural networks. You may need to run the following command to install the module.

```
1 pip install skorch
```

To use these wrappers, you must define a your PyTorch model as a class using `nn.Module`, then pass the name of the class to the `module` argument when constructing the `NeuralNetClassifier` class. For example:

```
1 class MyClassifier(nn.Module):
2     def __init__(self):
3         super().__init__()
4         ...
5
6     def forward(self, x):
7         ...
8         return x
9
10 # create the skorch wrapper
11 model = NeuralNetClassifier(
12     module=MyClassifier
13 )
```

The constructor for the `NeuralNetClassifier` class can take default arguments that are passed on to the calls to `model.fit()` (the way to invoke a training loop in scikit-learn models), such as the number of epochs and the batch size. For example:

```
1 model = NeuralNetClassifier(
2     module=MyClassifier,
3     max_epochs=150,
4     batch_size=10
5 )
```

The constructor for the `NeuralNetClassifier` class can also take new arguments that can be passed to your model class' constructor, but you have to prepend it with `module__` (with two underscores). These new arguments may carry a default value in the constructor but they will be overridden when the wrapper instantiate the model. For example:

```
1 import torch.nn as nn
2 from skorch import NeuralNetClassifier
3
4 class SonarClassifier(nn.Module):
5     def __init__(self, n_layers=3):
6         super().__init__()
7         self.layers = []
8         self.acts = []
9         for i in range(n_layers):
10             self.layers.append(nn.Linear(60, 60))
11             self.acts.append(nn.ReLU())
12             self.add_module(f"layer{i}", self.layers[-1])
13             self.add_module(f"act{i}", self.acts[-1])
14         self.output = nn.Linear(60, 1)
15
16     def forward(self, x):
17         for layer, act in zip(self.layers, self.acts):
18             x = act(layer(x))
19         x = self.output(x)
20         return x
21
22 model = NeuralNetClassifier(
23     module=SonarClassifier,
24     max_epochs=150,
25     batch_size=10,
26     module__n_layers=2
27 )
```

You can verify the result by initializing a model and print it:

```
1 print(model.initialize())
```

In this example, you should see:

```

1 <class 'skorch.classifier.NeuralNetClassifier'>[initialized](
2   module_=SonarClassifier(
3     (layer0): Linear(in_features=60, out_features=60, bias=True)
4     (act0): ReLU()
5     (layer1): Linear(in_features=60, out_features=60, bias=True)
6     (act1): ReLU()
7     (output): Linear(in_features=60, out_features=1, bias=True)
8   ),
9 )

```

Want to Get Started With Deep Learning with PyTorch?

Take my free email crash course now (with sample code).

Click to sign-up and also get a free PDF Ebook version of the course.

Download Your FREE Mini-Course

How to Use Grid Search in scikit-learn

Grid search is a model hyperparameter optimization technique. It simply exhaust all combinations of the hyperparameters and find the one that gave the best score. In scikit-learn, this technique is provided in the `GridSearchCV` class. When constructing this class, you must provide a dictionary of hyperparameters to evaluate in the `param_grid` argument. This is a map of the model parameter name and an array of values to try.

By default, accuracy is the score that is optimized, but other scores can be specified in the `score` argument of the `GridSearchCV` constructor. The `GridSearchCV` process will then construct and evaluate one model for each combination of parameters. Cross-validation is used to evaluate each individual model, and the default of 3-fold cross-validation is used, although you can override this by specifying the `cv` argument to the `GridSearchCV` constructor.

Below is an example of defining a simple grid search:

```

1 param_grid = {
2   'epochs': [10, 20, 30]
3 }
4 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
5 grid_result = grid.fit(X, Y)

```

By setting the `n_jobs` argument in the `GridSearchCV` constructor to `-1`, the process will use all cores on your machine. Otherwise the grid search process will only run in single thread, which is slower in the multi-core CPUs.

Once completed, you can access the outcome of the grid search in the result object returned from `grid.fit()`. The `best_score_` member provides access to the best score observed during the optimization procedure, and the `best_params_` describes the combination of parameters that achieved the best results. You can learn more about the `GridSearchCV` class in the scikit-learn API documentation.

Kick-start your project with my book [Deep Learning with PyTorch](#). It provides **self-study tutorials** with **working code**.

Problem Description

Now that you know how to use PyTorch models with scikit-learn and how to use grid search in scikit-learn, let's look at a bunch of examples.

All examples will be demonstrated on a small standard machine learning dataset called the [Pima Indians onset of diabetes classification dataset](#). This is a small dataset with all numerical attributes that is easy to work with.

As you proceed through the examples in this post, you will aggregate the best parameters. This is not the best way to grid search because parameters can interact, but it is good for demonstration purposes.

How to Tune Batch Size and Number of Epochs

In this first simple example, you will look at tuning the batch size and number of epochs used when fitting the network.

The batch size in iterative gradient descent is the number of patterns shown to the network before the weights are updated. It is also an optimization in the training of the network, defining how many patterns to read at a time and keep in memory.

The number of epochs is the number of times the entire training dataset is shown to the network during training. Some networks are sensitive to the batch size, such as LSTM recurrent neural networks and Convolutional Neural Networks.

Here you will evaluate a suite of different minibatch sizes from 10 to 100 in steps of 20.

The full code listing is provided below:

```

1 import random
2 import numpy as np
3 import torch
4 import torch.nn as nn
5 import torch.optim as optim
6 from skorch import NeuralNetClassifier
7 from sklearn.model_selection import GridSearchCV
8
9 # load the dataset, split into input (X) and output (y) variables
10 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
11 X = dataset[:,0:8]
12 y = dataset[:,8]
13 X = torch.tensor(X, dtype=torch.float32)
14 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
15
16 # PyTorch classifier
17 class PimaClassifier(nn.Module):
18     def __init__(self):
19         super().__init__()
20         self.layer = nn.Linear(8, 12)
21         self.act = nn.ReLU()
22         self.output = nn.Linear(12, 1)
23         self.prob = nn.Sigmoid()
24
25     def forward(self, x):
26         x = self.act(self.layer(x))
27         x = self.output(self.act(x))
28         return x
29
30 # create model with skorch
31 model = NeuralNetClassifier(
32     PimaClassifier,
33     criterion=nn.BCELoss,
34     optimizer=optim.Adam,
35     verbose=False
36 )
37
38 # define the grid search parameters
39 param_grid = {
40     'batch_size': [10, 20, 40, 60, 80, 100],
41     'max_epochs': [10, 50, 100]
42 }
43 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
44 grid_result = grid.fit(X, y)
45
46 # summarize results
47 print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
48 means = grid_result.cv_results_['mean_test_score']
49 stds = grid_result.cv_results_['std_test_score']
50 params = grid_result.cv_results_['params']
51 for mean, stdev, param in zip(means, stds, params):
52     print("%f (%f) with: %r" % (mean, stdev, param))

```

Running this example produces the following output:

```

1 Best: 0.714844 using {'batch_size': 10, 'max_epochs': 100}
2 0.665365 (0.020505) with: {'batch_size': 10, 'max_epochs': 10}
3 0.588542 (0.168055) with: {'batch_size': 10, 'max_epochs': 50}
4 0.714844 (0.032369) with: {'batch_size': 10, 'max_epochs': 100}
5 0.671875 (0.022326) with: {'batch_size': 20, 'max_epochs': 10}
6 0.696615 (0.008027) with: {'batch_size': 20, 'max_epochs': 50}
7 0.714844 (0.019918) with: {'batch_size': 20, 'max_epochs': 100}
8 0.666667 (0.009744) with: {'batch_size': 40, 'max_epochs': 10}
9 0.687500 (0.033603) with: {'batch_size': 40, 'max_epochs': 50}
10 0.707031 (0.024910) with: {'batch_size': 40, 'max_epochs': 100}
11 0.667969 (0.014616) with: {'batch_size': 60, 'max_epochs': 10}
12 0.694010 (0.036966) with: {'batch_size': 60, 'max_epochs': 50}
13 0.694010 (0.042473) with: {'batch_size': 60, 'max_epochs': 100}
14 0.670573 (0.023939) with: {'batch_size': 80, 'max_epochs': 10}
15 0.674479 (0.020752) with: {'batch_size': 80, 'max_epochs': 50}
16 0.703125 (0.026107) with: {'batch_size': 80, 'max_epochs': 100}
17 0.680990 (0.014382) with: {'batch_size': 100, 'max_epochs': 10}
18 0.670573 (0.013279) with: {'batch_size': 100, 'max_epochs': 50}
19 0.687500 (0.017758) with: {'batch_size': 100, 'max_epochs': 100}

```

You can see that the batch size of 10 and 100 epochs achieved the best result of about 71% accuracy (but you should also take into account the accuracy's standard deviation).

How to Tune the Training Optimization Algorithm

All deep learning library should offer a variety of optimization algorithms. PyTorch is no exception.

In this example, you will tune the optimization algorithm used to train the network, each with default parameters.

This is an odd example because often, you will choose one approach a priori and instead focus on tuning its parameters on your problem (see the next example).

Here, you will evaluate the suite of optimization algorithms available in PyTorch.

The full code listing is provided below:

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from skorch import NeuralNetClassifier
6 from sklearn.model_selection import GridSearchCV
7
8 # load the dataset, split into input (X) and output (y) variables
9 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
10 X = dataset[:,0:8]
11 y = dataset[:,8]
12 X = torch.tensor(X, dtype=torch.float32)
13 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
14
15 # PyTorch classifier
16 class PimaClassifier(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.layer = nn.Linear(8, 12)
20         self.act = nn.ReLU()
21         self.output = nn.Linear(12, 1)
22         self.proba = nn.Sigmoid()
23
24     def forward(self, x):
25         x = self.act(self.layer(x))
26         x = self.proba(self.output(x))
27         return x
28
29 # create model with skorch
30 model = NeuralNetClassifier(
31     PimaClassifier,
32     criterion=nn.BCELoss,
33     max_epochs=100,
34     batch_size=10,
35     verbose=False
36 )
37
38 # define the grid search parameters
39 param_grid = {
40     'optimizer': [optim.SGD, optim.RMSprop, optim.Adagrad, optim.Adadelta,
41                  optim.Adam, optim.Adamax, optim.NAdam],
42 }
43 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
44 grid_result = grid.fit(X, y)
45
46 # summarize results
47 print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
48 means = grid_result.cv_results_['mean_test_score']
49 stds = grid_result.cv_results_['std_test_score']
50 params = grid_result.cv_results_['params']
51 for mean, stdev, param in zip(means, stds, params):
52     print("%f (%f) with: %r" % (mean, stdev, param))
```

Running this example produces the following output:

```
1 Best: 0.721354 using {'optimizer': <class 'torch.optim.adamax.Adamax'>}
2 0.674479 (0.036828) with: {'optimizer': <class 'torch.optim.sgd.SGD'>}
3 0.700521 (0.043303) with: {'optimizer': <class 'torch.optim.rmsprop.RMSprop'>}
4 0.682292 (0.027126) with: {'optimizer': <class 'torch.optim.adagrad.Adagrad'>}
5 0.572917 (0.051560) with: {'optimizer': <class 'torch.optim.adadelta.Adadelta'>}
6 0.714844 (0.030758) with: {'optimizer': <class 'torch.optim.adam.Adam'>}
7 0.721354 (0.019225) with: {'optimizer': <class 'torch.optim.adamax.Adamax'>}
8 0.709635 (0.024360) with: {'optimizer': <class 'torch.optim.nadam.NAdam'>}
```

The results suggest that the Adamax optimization algorithm is the best with a score of about 72% accuracy.

It is worth to mention that GridSearchCV will recreate your model often so every trial is independent. The reason it can be done is because of the NeuralNetClassifier wrapper, which knows the name of the class for your PyTorch model and instantiate one for you upon request.

How to Tune Learning Rate and Momentum

It is common to pre-select an optimization algorithm to train your network and tune its parameters.

By far, the most common optimization algorithm is plain old Stochastic Gradient Descent (SGD) because it is so well understood. In this example, you will look at optimizing the SGD learning rate and momentum parameters.

The learning rate controls how much to update the weight at the end of each batch, and the momentum controls how much to let the previous update influence the current weight update.

You will try a suite of small standard learning rates and a momentum values from 0.2 to 0.8 in steps of 0.2, as well as 0.9 (because it can be a popular value in practice). In PyTorch, the way to set the learning rate and momentum is the following:

```
1 optimizer = optim.SGD(lr=0.001, momentum=0.9)
```

In the skorch wrapper, you will can route the parameters to the optimizer with the prefix optimizer__.

Generally, it is a good idea to also include the number of epochs in an optimization like this as there is a dependency between the amount of learning per batch (learning rate), the number of updates per epoch (batch size), and the number of epochs.

The full code listing is provided below:

```
1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.optim as optim
5 from skorch import NeuralNetClassifier
6 from sklearn.model_selection import GridSearchCV
7
8 # load the dataset, split into input (X) and output (y) variables
9 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
10 X = dataset[:,0:8]
11 y = dataset[:,8]
12 X = torch.tensor(X, dtype=torch.float32)
13 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
14
15 # PyTorch classifier
16 class PimaClassifier(nn.Module):
17     def __init__(self):
18         super().__init__()
19         self.layer = nn.Linear(8, 12)
20         self.act = nn.ReLU()
21         self.output = nn.Linear(12, 1)
22         self.prob = nn.Sigmoid()
23
24     def forward(self, x):
25         x = self.act(self.layer(x))
26         x = self.output(self.act(x))
27         return x
28
29 # create model with skorch
30 model = NeuralNetClassifier(
31     PimaClassifier,
32     criterion=nn.BCELoss,
33     optimizer=optim.SGD,
34     max_epochs=100,
35     batch_size=10,
36     verbose=False
37 )
38
39 # define the grid search parameters
40 param_grid = {
41     'optimizer__lr': [0.001, 0.01, 0.1, 0.2, 0.3],
42     'optimizer__momentum': [0.0, 0.2, 0.4, 0.6, 0.8, 0.9],
43 }
44 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
45 grid_result = grid.fit(X, y)
46
47 # summarize results
48 print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
49 means = grid_result.cv_results_['mean_test_score']
50 stds = grid_result.cv_results_['std_test_score']
51 params = grid_result.cv_results_['params']
52 for mean, stdev, param in zip(means, stds, params):
53     print("%f (%f) with: %r" % (mean, stdev, param))
```

Running this example produces the following output.

```
1 Best: 0.682292 using {'optimizer__lr': 0.001, 'optimizer__momentum': 0.9}
2 0.648438 (0.016877) with: {'optimizer__lr': 0.001, 'optimizer__momentum': 0.0}
3 0.671875 (0.017758) with: {'optimizer__lr': 0.001, 'optimizer__momentum': 0.2}
4 0.674479 (0.022402) with: {'optimizer__lr': 0.001, 'optimizer__momentum': 0.4}
5 0.677083 (0.011201) with: {'optimizer__lr': 0.001, 'optimizer__momentum': 0.6}
6 0.679688 (0.027621) with: {'optimizer__lr': 0.001, 'optimizer__momentum': 0.8}
7 0.682292 (0.026557) with: {'optimizer__lr': 0.001, 'optimizer__momentum': 0.9}
8 0.671875 (0.019918) with: {'optimizer__lr': 0.01, 'optimizer__momentum': 0.0}
9 0.648438 (0.024910) with: {'optimizer__lr': 0.01, 'optimizer__momentum': 0.2}
10 0.546875 (0.143454) with: {'optimizer__lr': 0.01, 'optimizer__momentum': 0.4}
11 0.567708 (0.153668) with: {'optimizer__lr': 0.01, 'optimizer__momentum': 0.6}
12 0.552083 (0.141790) with: {'optimizer__lr': 0.01, 'optimizer__momentum': 0.8}
13 0.451823 (0.144561) with: {'optimizer__lr': 0.01, 'optimizer__momentum': 0.9}
14 0.348958 (0.001841) with: {'optimizer__lr': 0.1, 'optimizer__momentum': 0.0}
15 0.450521 (0.142719) with: {'optimizer__lr': 0.1, 'optimizer__momentum': 0.2}
16 0.450521 (0.142719) with: {'optimizer__lr': 0.1, 'optimizer__momentum': 0.4}
17 0.450521 (0.142719) with: {'optimizer__lr': 0.1, 'optimizer__momentum': 0.6}
18 0.348958 (0.001841) with: {'optimizer__lr': 0.1, 'optimizer__momentum': 0.8}
19 0.348958 (0.001841) with: {'optimizer__lr': 0.1, 'optimizer__momentum': 0.9}
```



```

20 0.444010 (0.136265) with: {'optimizer__lr': 0.2, 'optimizer__momentum': 0.0}
21 0.450521 (0.142719) with: {'optimizer__lr': 0.2, 'optimizer__momentum': 0.2}
22 0.348958 (0.001841) with: {'optimizer__lr': 0.2, 'optimizer__momentum': 0.4}
23 0.552083 (0.141790) with: {'optimizer__lr': 0.2, 'optimizer__momentum': 0.6}
24 0.549479 (0.142719) with: {'optimizer__lr': 0.2, 'optimizer__momentum': 0.8}
25 0.651042 (0.001841) with: {'optimizer__lr': 0.2, 'optimizer__momentum': 0.9}
26 0.552083 (0.141790) with: {'optimizer__lr': 0.3, 'optimizer__momentum': 0.0}
27 0.348958 (0.001841) with: {'optimizer__lr': 0.3, 'optimizer__momentum': 0.2}
28 0.450521 (0.142719) with: {'optimizer__lr': 0.3, 'optimizer__momentum': 0.4}
29 0.552083 (0.141790) with: {'optimizer__lr': 0.3, 'optimizer__momentum': 0.6}
30 0.450521 (0.142719) with: {'optimizer__lr': 0.3, 'optimizer__momentum': 0.8}
31 0.450521 (0.142719) with: {'optimizer__lr': 0.3, 'optimizer__momentum': 0.9}

```

You can see that, with SGD, the best results were achieved using a learning rate of 0.001 and a momentum of 0.9 with an accuracy of about 68%.

How to Tune Network Weight Initialization

Neural network weight initialization used to be simple: use small random values.

Now there is a suite of different techniques to choose from. You can get a laundry list from `torch.nn.init` documentation.

In this example, you will look at tuning the selection of network weight initialization by evaluating all the available techniques.

You will use the same weight initialization method on each layer. Ideally, it may be better to use different weight initialization schemes according to the activation function used on each layer. In the example below, you will use a rectifier for the hidden layer. Use sigmoid for the output layer because the predictions are binary. The weight initialization is implicit in PyTorch models. Therefore you need to write your own logic to initialize the weight, after the layer is created but before it is used. Let's modify the PyTorch as follows:

```

1 # PyTorch classifier
2 class PimaClassifier(nn.Module):
3     def __init__(self, weight_init=torch.nn.init.xavier_uniform_):
4         super().__init__()
5         self.layer = nn.Linear(8, 12)
6         self.act = nn.ReLU()
7         self.output = nn.Linear(12, 1)
8         self.prob = nn.Sigmoid()
9         # manually init weights
10        weight_init(self.layer.weight)
11        weight_init(self.output.weight)
12
13    def forward(self, x):
14        x = self.act(self.layer(x))
15        x = self.prob(self.output(x))
16        return x

```

An argument `weight_init` is added to the class `PimaClassifier` and it expects one of the initializers from `torch.nn.init`. In `GridSearchCV`, you need to use the `module__` prefix to make `NeuralNetClassifier` route the parameter to the model's class constructor.

The full code listing is provided below:

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.nn.init as init
5 import torch.optim as optim
6 from skorch import NeuralNetClassifier
7 from sklearn.model_selection import GridSearchCV
8
9 # load the dataset, split into input (X) and output (y) variables
10 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
11 X = dataset[:,0:8]
12 y = dataset[:,8]
13 X = torch.tensor(X, dtype=torch.float32)
14 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
15
16 # PyTorch classifier
17 class PimaClassifier(nn.Module):
18     def __init__(self, weight_init=init.xavier_uniform_):
19         super().__init__()
20         self.layer = nn.Linear(8, 12)
21         self.act = nn.ReLU()
22         self.output = nn.Linear(12, 1)
23         self.prob = nn.Sigmoid()
24         # manually init weights
25         weight_init(self.layer.weight)
26         weight_init(self.output.weight)
27
28     def forward(self, x):
29         x = self.act(self.layer(x))
30         x = self.prob(self.output(x))
31         return x
32
33 # create model with skorch
34 model = NeuralNetClassifier(
35     PimaClassifier,
36     criterion=nn.BCELoss,
37     optimizer=optim.Adamax,

```

```

38     max_epochs=100,
39     batch_size=10,
40     verbose=False
41 )
42
43 # define the grid search parameters
44 param_grid = {
45     'module__weight_init': [init.uniform_, init.normal_, init.zeros_,
46                             init.xavier_normal_, init.xavier_uniform_,
47                             init.kaiming_normal_, init.kaiming_uniform_]
48 }
49 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
50 grid_result = grid.fit(X, y)
51
52 # summarize results
53 print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
54 means = grid_result.cv_results_['mean_test_score']
55 stds = grid_result.cv_results_['std_test_score']
56 params = grid_result.cv_results_['params']
57 for mean, stdev, param in zip(means, stds, params):
58     print("%f (%f) with: %r" % (mean, stdev, param))

```

Running this example produces the following output.

```

1 Best: 0.697917 using {'module__weight_init': <function kaiming_uniform_ at 0x112020c10>}
2 0.348958 (0.001841) with: {'module__weight_init': <function uniform_ at 0x1120204c0>}
3 0.602865 (0.061708) with: {'module__weight_init': <function normal_ at 0x112020550>}
4 0.652344 (0.003189) with: {'module__weight_init': <function zeros_ at 0x112020820>}
5 0.691406 (0.030758) with: {'module__weight_init': <function xavier_normal_ at 0x112020af0>}
6 0.592448 (0.171589) with: {'module__weight_init': <function xavier_uniform_ at 0x112020a60>}
7 0.563802 (0.152971) with: {'module__weight_init': <function kaiming_normal_ at 0x112020ca0>}
8 0.697917 (0.013279) with: {'module__weight_init': <function kaiming_uniform_ at 0x112020c10>}

```

The best results were achieved with a He-uniform weight initialization scheme achieving a performance of about 70%.

How to Tune the Neuron Activation Function

The activation function controls the nonlinearity of individual neurons and when to fire.

Generally, the rectifier activation function is the most popular. However, it used to be the sigmoid and the tanh functions, and these functions may still be more suitable for different problems.

In this example, you will evaluate some of the activation functions available in PyTorch. You will only use these functions in the hidden layer, as a sigmoid activation function is required in the output for the binary classification problem. Similar to the previous example, this is an argument to the class constructor of the model, and you will use the `module__` prefix for the GridSearchCV parameter grid.

Generally, it is a good idea to prepare data to the range of the different transfer functions, which you will not do in this case.

The full code listing is provided below:

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.nn.init as init
5 import torch.optim as optim
6 from skorch import NeuralNetClassifier
7 from sklearn.model_selection import GridSearchCV
8
9 # load the dataset, split into input (X) and output (y) variables
10 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
11 X = dataset[:,0:8]
12 y = dataset[:,8]
13 X = torch.tensor(X, dtype=torch.float32)
14 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
15
16 # PyTorch classifier
17 class PimaClassifier(nn.Module):
18     def __init__(self, activation=nn.ReLU):
19         super().__init__()
20         self.layer = nn.Linear(8, 12)
21         self.act = activation()
22         self.output = nn.Linear(12, 1)
23         self.prob = nn.Sigmoid()
24         # manually init weights
25         init.kaiming_uniform_(self.layer.weight)
26         init.kaiming_uniform_(self.output.weight)
27
28     def forward(self, x):
29         x = self.act(self.layer(x))
30         x = self.prob(self.output(x))
31         return x
32
33 # create model with skorch
34 model = NeuralNetClassifier(
35     PimaClassifier,
36     criterion=nn.BCELoss,
37     optimizer=optim.Adamax,
38     max_epochs=100,

```



```

39     batch_size=10,
40     verbose=False
41 )
42
43 # define the grid search parameters
44 param_grid = {
45     'module__activation': [nn.Identity, nn.ReLU, nn.ELU, nn.ReLU6,
46                           nn.GELU, nn.Softplus, nn.Softsign, nn.Tanh,
47                           nn.Sigmoid, nn.Hardsigmoid]
48 }
49 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
50 grid_result = grid.fit(X, y)
51
52 # summarize results
53 print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
54 means = grid_result.cv_results_['mean_test_score']
55 stds = grid_result.cv_results_['std_test_score']
56 params = grid_result.cv_results_['params']
57 for mean, stdev, param in zip(means, stds, params):
58     print("%f (%f) with: %r" % (mean, stdev, param))

```

Running this example produces the following output.

```

1 Best: 0.699219 using {'module__activation': <class 'torch.nn.modules.activation.ReLU'>}
2 0.687500 (0.025315) with: {'module__activation': <class 'torch.nn.modules.linear.Identity'>}
3 0.699219 (0.011049) with: {'module__activation': <class 'torch.nn.modules.activation.ReLU'>}
4 0.674479 (0.035849) with: {'module__activation': <class 'torch.nn.modules.activation.ELU'>}
5 0.621094 (0.063549) with: {'module__activation': <class 'torch.nn.modules.activation.ReLU6'>}
6 0.674479 (0.017566) with: {'module__activation': <class 'torch.nn.modules.activation.GELU'>}
7 0.558594 (0.149189) with: {'module__activation': <class 'torch.nn.modules.activation.Softplus'>}
8 0.675781 (0.014616) with: {'module__activation': <class 'torch.nn.modules.activation.Softsign'>}
9 0.619792 (0.018688) with: {'module__activation': <class 'torch.nn.modules.activation.Tanh'>}
10 0.643229 (0.019225) with: {'module__activation': <class 'torch.nn.modules.activation.Sigmoid'>}
11 0.636719 (0.022326) with: {'module__activation': <class 'torch.nn.modules.activation.Hardsigmoid'>}

```

It shows that ReLU activation function achieved the best results with an accuracy of about 70%.

How to Tune Dropout Regularization

In this example, you will look at tuning the dropout rate for regularization in an effort to limit overfitting and improve the model's ability to generalize.

For the best results, dropout is best combined with a weight constraint such as the max norm constraint, which is implemented in the forward pass function.

This involves fitting both the dropout percentage and the weight constraint. We will try dropout percentages between 0.0 and 0.9 (1.0 does not make sense) and MaxNorm weight constraint values between 0 and 5.

The full code listing is provided below.

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.nn.init as init
5 import torch.optim as optim
6 from skorch import NeuralNetClassifier
7 from sklearn.model_selection import GridSearchCV
8
9 # load the dataset, split into input (X) and output (y) variables
10 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
11 X = dataset[:,0:8]
12 y = dataset[:,8]
13 X = torch.tensor(X, dtype=torch.float32)
14 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
15
16 # PyTorch classifier
17 class PimaClassifier(nn.Module):
18     def __init__(self, dropout_rate=0.5, weight_constraint=1.0):
19         super().__init__()
20         self.layer = nn.Linear(8, 12)
21         self.act = nn.ReLU()
22         self.dropout = nn.Dropout(dropout_rate)
23         self.output = nn.Linear(12, 1)
24         self.prob = nn.Sigmoid()
25         self.weight_constraint = weight_constraint
26         # manually init weights
27         init.kaiming_uniform_(self.layer.weight)
28         init.kaiming_uniform_(self.output.weight)
29
30     def forward(self, x):
31         # maxnorm weight before actual forward pass
32         with torch.no_grad():
33             norm = self.layer.weight.norm(2, dim=0, keepdim=True).clamp(min=self.weight_constraint / 2)
34             desired = torch.clamp(norm, max=self.weight_constraint)
35             self.layer.weight *= (desired / norm)
36         # actual forward pass
37         x = self.act(self.layer(x))
38         x = self.dropout(x)
39         x = self.prob(self.output(x))
40         return x

```

```

41
42 # create model with skorch
43 model = NeuralNetClassifier(
44     PimaClassifier,
45     criterion=nn.BCELoss,
46     optimizer=optim.Adamax,
47     max_epochs=100,
48     batch_size=10,
49     verbose=False
50 )
51
52 # define the grid search parameters
53 param_grid = {
54     'module__weight_constraint': [1.0, 2.0, 3.0, 4.0, 5.0],
55     'module__dropout_rate': [0.0, 0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9]
56 }
57 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
58 grid_result = grid.fit(X, y)
59
60 # summarize results
61 print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
62 means = grid_result.cv_results_['mean_test_score']
63 stds = grid_result.cv_results_['std_test_score']
64 params = grid_result.cv_results_['params']
65 for mean, stdev, param in zip(means, stds, params):
66     print("%f (%f) with: %r" % (mean, stdev, param))

```

Running this example produces the following output.

```

1 Best: 0.701823 using {'module__dropout_rate': 0.1, 'module__weight_constraint': 2.0}
2 0.669271 (0.015073) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 1.0}
3 0.692708 (0.035132) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 2.0}
4 0.589844 (0.170180) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 3.0}
5 0.561198 (0.151131) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 4.0}
6 0.688802 (0.021710) with: {'module__dropout_rate': 0.0, 'module__weight_constraint': 5.0}
7 0.697917 (0.009744) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 1.0}
8 0.701823 (0.016367) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 2.0}
9 0.694010 (0.010253) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 3.0}
10 0.686198 (0.025976) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 4.0}
11 0.679688 (0.026107) with: {'module__dropout_rate': 0.1, 'module__weight_constraint': 5.0}
12 0.701823 (0.029635) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 1.0}
13 0.682292 (0.014731) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 2.0}
14 0.701823 (0.009744) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 3.0}
15 0.701823 (0.026557) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 4.0}
16 0.687500 (0.015947) with: {'module__dropout_rate': 0.2, 'module__weight_constraint': 5.0}
17 0.686198 (0.006639) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 1.0}
18 0.656250 (0.006379) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 2.0}
19 0.565104 (0.155608) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 3.0}
20 0.700521 (0.028940) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 4.0}
21 0.669271 (0.012890) with: {'module__dropout_rate': 0.3, 'module__weight_constraint': 5.0}
22 0.661458 (0.018688) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 1.0}
23 0.669271 (0.017566) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 2.0}
24 0.652344 (0.006379) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 3.0}
25 0.680990 (0.037783) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 4.0}
26 0.692708 (0.042112) with: {'module__dropout_rate': 0.4, 'module__weight_constraint': 5.0}
27 0.666667 (0.006639) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 1.0}
28 0.652344 (0.011500) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 2.0}
29 0.662760 (0.007366) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 3.0}
30 0.558594 (0.146610) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 4.0}
31 0.552083 (0.141826) with: {'module__dropout_rate': 0.5, 'module__weight_constraint': 5.0}
32 0.548177 (0.141826) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 1.0}
33 0.653646 (0.013279) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 2.0}
34 0.661458 (0.008027) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 3.0}
35 0.553385 (0.142719) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 4.0}
36 0.669271 (0.035132) with: {'module__dropout_rate': 0.6, 'module__weight_constraint': 5.0}
37 0.662760 (0.015733) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 1.0}
38 0.636719 (0.024910) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 2.0}
39 0.550781 (0.146818) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 3.0}
40 0.537760 (0.140094) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 4.0}
41 0.542969 (0.138144) with: {'module__dropout_rate': 0.7, 'module__weight_constraint': 5.0}
42 0.565104 (0.148654) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 1.0}
43 0.657552 (0.008027) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 2.0}
44 0.428385 (0.111418) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 3.0}
45 0.549479 (0.142719) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 4.0}
46 0.648438 (0.005524) with: {'module__dropout_rate': 0.8, 'module__weight_constraint': 5.0}
47 0.540365 (0.136861) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 1.0}
48 0.605469 (0.053083) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 2.0}
49 0.553385 (0.139948) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 3.0}
50 0.549479 (0.142719) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 4.0}
51 0.595052 (0.075566) with: {'module__dropout_rate': 0.9, 'module__weight_constraint': 5.0}

```

You can see that the dropout rate of 10% and the weight constraint of 2.0 resulted in the best accuracy of about 70%.

How to Tune the Number of Neurons in the Hidden Layer

The number of neurons in a layer is an important parameter to tune. Generally the number of neurons in a layer controls the representational capacity of the network, at least at that point in the topology.

Generally, a large enough single layer network can approximate any other neural network, due to the [universal approximation theorem](#).

In this example, you will look at tuning the number of neurons in a single hidden layer. you will try values from 1 to 30 in steps of 5.

A larger network requires more training and at least the batch size and number of epochs should ideally be optimized with the number of neurons.

The full code listing is provided below.

```

1 import numpy as np
2 import torch
3 import torch.nn as nn
4 import torch.nn.init as init
5 import torch.optim as optim
6 from skorch import NeuralNetClassifier
7 from sklearn.model_selection import GridSearchCV
8
9 # load the dataset, split into input (X) and output (y) variables
10 dataset = np.loadtxt('pima-indians-diabetes.csv', delimiter=',')
11 X = dataset[:,0:8]
12 y = dataset[:,8]
13 X = torch.tensor(X, dtype=torch.float32)
14 y = torch.tensor(y, dtype=torch.float32).reshape(-1, 1)
15
16 class PimaClassifier(nn.Module):
17     def __init__(self, n_neurons=12):
18         super().__init__()
19         self.layer = nn.Linear(8, n_neurons)
20         self.act = nn.ReLU()
21         self.dropout = nn.Dropout(0.1)
22         self.output = nn.Linear(n_neurons, 1)
23         self.prob = nn.Sigmoid()
24         self.weight_constraint = 2.0
25         # manually init weights
26         init.kaiming_uniform_(self.layer.weight)
27         init.kaiming_uniform_(self.output.weight)
28
29     def forward(self, x):
30         # maxnorm weight before actual forward pass
31         with torch.no_grad():
32             norm = self.layer.weight.norm(2, dim=0, keepdim=True).clamp(min=self.weight_constraint / 2)
33             desired = torch.clamp(norm, max=self.weight_constraint)
34             self.layer.weight *= (desired / norm)
35         # actual forward pass
36         x = self.act(self.layer(x))
37         x = self.dropout(x)
38         x = self.prob(self.output(x))
39         return x
40
41 # create model with skorch
42 model = NeuralNetClassifier(
43     PimaClassifier,
44     criterion=nn.BCELoss,
45     optimizer=optim.Adamax,
46     max_epochs=100,
47     batch_size=10,
48     verbose=False
49 )
50
51 # define the grid search parameters
52 param_grid = {
53     'module__n_neurons': [1, 5, 10, 15, 20, 25, 30]
54 }
55 grid = GridSearchCV(estimator=model, param_grid=param_grid, n_jobs=-1, cv=3)
56 grid_result = grid.fit(X, y)
57
58 # summarize results
59 print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))
60 means = grid_result.cv_results_['mean_test_score']
61 stds = grid_result.cv_results_['std_test_score']
62 params = grid_result.cv_results_['params']
63 for mean, stdev, param in zip(means, stds, params):
64     print("%f (%f) with: %r" % (mean, stdev, param))

```

Running this example produces the following output.

```

1 Best: 0.708333 using {'module__n_neurons': 30}
2 0.654948 (0.003683) with: {'module__n_neurons': 1}
3 0.666667 (0.023073) with: {'module__n_neurons': 5}
4 0.694010 (0.014382) with: {'module__n_neurons': 10}
5 0.682292 (0.014382) with: {'module__n_neurons': 15}
6 0.707031 (0.028705) with: {'module__n_neurons': 20}
7 0.703125 (0.030758) with: {'module__n_neurons': 25}
8 0.708333 (0.015733) with: {'module__n_neurons': 30}

```

You can see that the best results were achieved with a network with 30 neurons in the hidden layer with an accuracy of about 71%.

Tips for Hyperparameter Optimization

This section lists some handy tips to consider when tuning hyperparameters of your neural network.

- **k-Fold Cross-Validation.** You can see that the results from the examples in this post show some variance. A default cross-validation of 3 was used, but perhaps $k = 5$ or $k = 10$ would be more stable. Carefully choose your cross-validation configuration to ensure your results are

stable.

- **Review the Whole Grid.** Do not just focus on the best result, review the whole grid of results and look for trends to support configuration decisions. Of course, there will be more combinations and it takes a longer time to evaluate.
- **Parallelize.** Use all your cores if you can, neural networks are slow to train and we often want to try a lot of different parameters. Consider to run it on a cloud platform, such as AWS.
- **Use a Sample of Your Dataset.** Because networks are slow to train, try training them on a smaller sample of your training dataset, just to get an idea of general directions of parameters rather than optimal configurations.
- **Start with Coarse Grids.** Start with coarse-grained grids and zoom into finer grained grids once you can narrow the scope.
- **Do Not Transfer Results.** Results are generally problem specific. Try to avoid favorite configurations on each new problem that you see. It is unlikely that optimal results you discover on one problem will transfer to your next project. Instead look for broader trends like number of layers or relationships between parameters.
- **Reproducibility is a Problem.** Although we set the seed for the random number generator in NumPy, the results are not 100% reproducible. There is more to reproducibility when grid searching wrapped PyTorch models than is presented in this post.

Further Reading

This section provides more resources on the topic if you are looking to go deeper.

- [skorch documentation](#)
- [torch.nn](#) from PyTorch
- [GridSearchCV](#) from scikit-learn

Summary

In this post, you discovered how you can tune the hyperparameters of your deep learning networks in Python using PyTorch and scikit-learn. Specifically, you learned:

- How to wrap PyTorch models for use in scikit-learn and how to use grid search.
- How to grid search a suite of different standard neural network parameters for PyTorch models.
- How to design your own hyperparameter optimization experiments.

Get Started on Deep Learning with PyTorch!

Learn how to build deep learning models

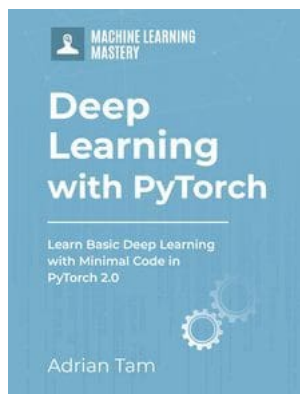
...using the newly released PyTorch 2.0 library

Discover how in my new Ebook:
Deep Learning with PyTorch

It provides **self-study tutorials** with **hundreds of working code** to turn you from a novice to expert. It equips you with *tensor operation, training, evaluation, hyperparameter optimization*, and much more...

Kick-start your deep learning journey with hands-on exercises

SEE WHAT'S INSIDE

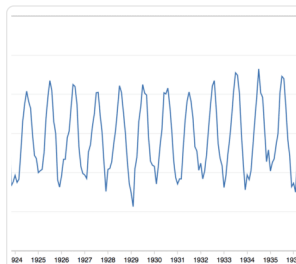
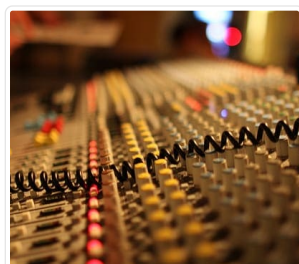


Share

Share

Post

More On This Topic

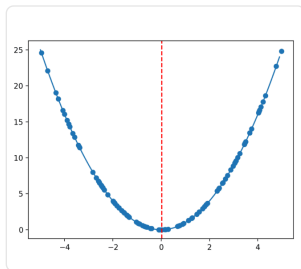


How to Grid Search Hyperparameters for Deep Learning...

How to Grid Search ARIMA Model Hyperparameters with Python

How to Grid Search SARIMA Hyperparameters for Time...

Hyperparameter Optimization With Random Search and...



Random Search and Grid Search for Function Optimization

How to Grid Search Deep Learning Models for Time...



About Adrian Tam

Adrian Tam, PhD is a data scientist and software engineer.

[View all posts by Adrian Tam →](#)

[< Use PyTorch Deep Learning Models with scikit-learn](#)

[Save and Load Your PyTorch Models >](#)

6 Responses to *How to Grid Search Hyperparameters for PyTorch Models*



Aminul October 12, 2023 at 5:49 am #

REPLY ↩

Hi,

In this scenario, can I use pytorch models with GPU support?



James Carmichael October 12, 2023 at 8:28 am #

REPLY ↩

Hi Aminul...The following resource is a great starting point for utilizing GPUs in Pytorch.

<https://medium.com/ai%C2%B3-theory-practice-business/use-gpu-in-your-pytorch-code-676a67faed09>



Yaswanth November 22, 2023 at 3:33 am #

REPLY ↩

hello ,

The code is probing an error at grid.fit(x,y) line. what might be the error ?



James Carmichael November 22, 2023 at 10:30 am #

REPLY ↩

Hi Yaswanth...Please let us know the exact verbiage of the error you are encountering. That will better enable us to assist you.



Lukas January 23, 2024 at 1:22 am #

REPLY ↩

Hi,

I just wanted to say that the guides here on machinelearningmastery.com have been incredibly valuable for me while studying machine learning. I've only been working with it for 2 years now but im excited about whats to come.

Thank you so much.

All the best



James Carmichael January 23, 2024 at 9:17 am #

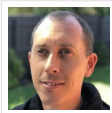
REPLY ↩

Thank you for your feedback Lukas! We appreciate the support!

Leave a Reply

 Name (required) Email (will not be published) (required)

SUBMIT COMMENT



Welcome!

I'm *Jason Brownlee* PhD

and I **help developers** get results with **machine learning**.

[Read more](#)

Never miss a tutorial:



Picked for you:



PyTorch Tutorial: How to Develop Deep Learning Models with Python



LSTM for Time Series Prediction in PyTorch



Deep Learning with PyTorch (9-Day Mini-Course)

[Calculating Derivatives in PyTorch](#)[Develop Your First Neural Network with PyTorch, Step by Step](#)

Loving the Tutorials?

The Deep Learning with PyTorch EBook is where you'll find the **Really Good** stuff.

>> SEE WHAT'S INSIDE



Machine Learning Mastery is part of Guiding Tech Media, a leading digital media publisher focused on helping people figure out technology. [Visit our corporate website](#) to learn more about our mission and team.



[PRIVACY](#) | [DISCLAIMER](#) | [TERMS](#) | [CONTACT](#) | [SITEMAP](#) | [ADVERTISE WITH US](#)

© 2026 Guiding Tech Media All Rights Reserved